
Cyberboswachters

De beste digitale stropers zijn ook de beste cyberboswachters

Dams Tim



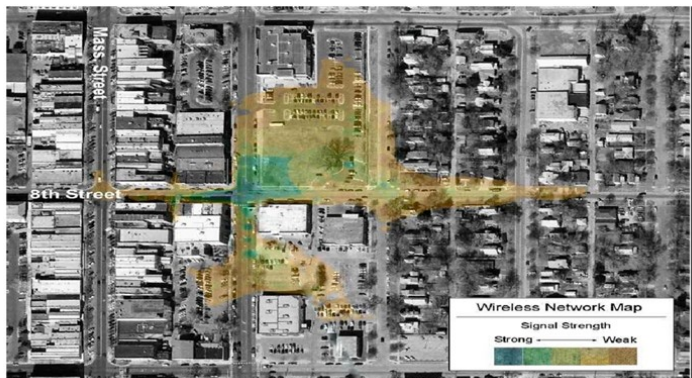
Wifi security

Alhoewel dit hoofdstuk integraal na het crypto hoofdstuk komt, is het toch interessant om dit hoofdstuk geschrinkt met het crypto hoofdstuk door te nemen als volgt:

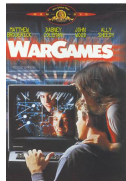
- “Crypto” tot en met “Symmetric stream ciphers”.
- “Wifi security” tot en met “WEP en waarom het faalde”.
- De rest van “Crypto”.
- De rest van “Wifi security”

De problemen van Wifi

We kunnen draadloze netwerken, specifiek wifi-netwerken, niet meer uit ons leven inbeelden. De opkomst van de IEEE 802.11b standaard (spreek IEEE uit als “*Ai-trippel-i*”) in 1999 veroorzaakte een kleine revolutie in de manier waarop bedrijven en privégebruikers konden werken. Plots kon je met een laptop van overal in het gebouw - en zelfs er buiten- op het netwerk geraken. Die vrijheid voor de gebruikers betekende wel een nachtmerrie voor de cyberboswachters. Een netwerkkabel heeft een intrinsieke extra beveiliging: enkel daar waar de kabel ligt kunnen gebruikers aan het netwerk geraken. Zolang je dus geen netwerkkabel bijvoorbeeld naar de publieke parking brengt kan niemand van daar illegaal het netwerk benaderen. Met wifi leek het alsof plotseling het hele netwerk in een straal van tientallen meters rond het gebouw beschikbaar was, met alle gevolgen van dien.



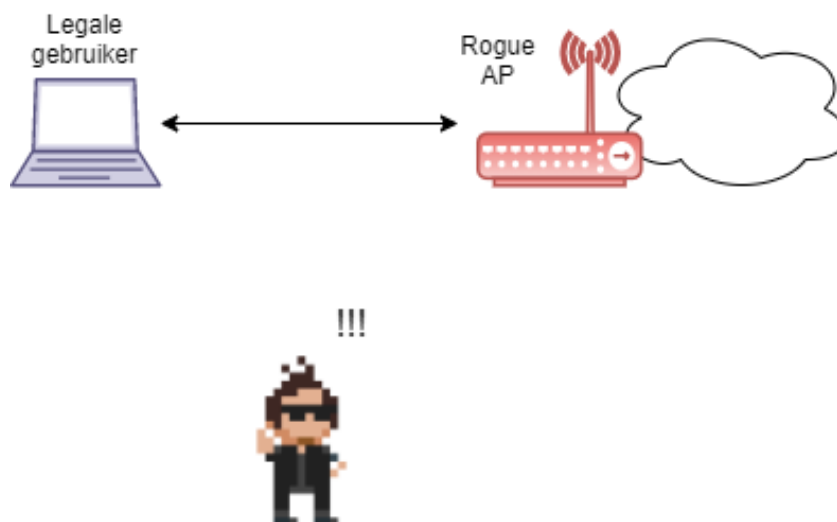
Al gauw werd een nieuwe sport uitgevonden door hobbyist hackers en professionele cybercriminelen: **wardriving**. De idee is eenvoudig: rijdt rond in de stad en laat de laptop naast je in de auto scannen naar alle netwerken, met extra aandacht voor die netwerken die geen of zwakke beveiliging hadden.



De term wardriving komt van de term *wardialing* die op zijn beurt gebaseerd is op de klassieke cyber-cult film “Wargames” uit 1983. Voor de geschiedkundigen onder ons, wardialing was het opbellen van willekeurige telefoonnummers met je modem in de hoop een zogenaamd *bulletin board system* oftewel **BBS** (een pre-internet forum zeg maar) te vinden.

Draadloze netwerken die gevonden worden hebben dan ook een schare aan problemen:

- **Eavesdropping:** iedereen kan “meeluisteren” wat er door de lucht vliegt. Dit heb je niet met een bedrade kabel.
- **Invasion:** je kan eenvoudig verbinden met het netwerk indien er geen beveiliging voorzien werd. Iets dat je dus bij een bedraad netwerk enkel kunt als je fysiek toegang hebt tot een netwerkkabel.
- **Man-in-the-middle aanvallen:** hier gaan we zo meteen dieper op in.
- **Backdoor:** een veelvoorkomend probleem zijn zogenaamde **rogue access points** die worden bijgeplaatst op het netwerk door goedbedoelde werknemers die zou het bereik van het netwerk wat willen uitbreiden. Vaak zijn echter de beveiligingsinstellingen van zo’n (meestal goedkoop) access point niet zo sterk als die van het bedrijf. Bijgevolg is dit de ideale backdoor voor malafide gebruikers die het netwerk aan het surveilleren zijn. Wanneer ze een netwerkscan doen zullen ze tientallen goed beveiligde access points zien en 1 zwak beveiligd.



Figuur 0.1: Een rogue access point is de ideale manier om binnen te geraken voor hacker.

- **Denial-of-service op fysiek niveau:** om een gebruiker toegang tot een bedraad netwerk te ontfeggen op fysiek niveau dien je de kabel door te knippen. Bij wifi is dit nog eenvoudiger: de “kabel” bij wifi zijn de frequentiebanden in de lucht waarbinnen de apparaten mogen werken (circa 2.4 Ghz bij de oudere wifi apparaten, nu meestal rond de 5 Ghz band). De wifi-apparaten kunnen enkel met elkaar communiceren indien zij een signaal naar elkaar over die frequentieband kunnen sturen op een moment dat niemand anders in de buurt die band gebruikt (zie note hierna). Als een malafide gebruiker dus die frequentieband vult met andere signalen, dan zullen de legale gebruikers nooit iets kunnen uitsturen. Wil je dus een wifi netwerk *Dos’n*, koop dan een signaalgenerator die op de juiste frequentieband de nodige ruist uitzend en klaar is kees.



Bedrade netwerk apparaten werken volgens het CSMA/CD (Carrier Sense Multiple Access / Collision Detection) om met elkaar over de draad te communiceren, hierbij detecteren ze wanneer er ‘botsingen’ tussen pakketen voordoen en de informatie dus opnieuw moet uitgestuurd worden. Bij draadloze netwerken is het echter onmogelijk om *botsingen in de lucht* te detecteren, daarom werken ze met een variant: **CSMA/CA**, oftewel CSMA/ **Collision Avoidance**. Een wifi-apparaat dat iets wil uitsturen zal eerst controleren of de frequentieband waarop ze werken vrij is, enkel dan zal er vervolgens een pakketje de lucht worden ingestuurd.



Sommige gebruikers schakelen het broadcasten van hun netwerknaam (het zogenaamde **SSID**) uit in de hoop dat zo dat burens, voorbijgangers en wardrivers hun netwerk niet kunnen zien. Helaas is dit zogenaamde *snake's oil*: het geeft een vals gevoel van veiligheid. Je kan weliswaar SSID-broadcasting uitzetten (dit is een pakketje dat je access point elke paar seconden uitstuurt om aan iedereen die luistert te zeggen “*Hallo, hier is het netwerk met naam X, en dit zijn de parameters die je nodig hebt om met mij te verbinden*”) maar het SSID wordt ook in een hoop andere pakketjes de lucht in gestuurd. Een beetje wifi hacker kan dus het SSID ogenblikkelijk uit de lucht plukken, ongeacht dat broadcasting werd uitgeschakeld of niet.



Om de huidige, en betere, beveiliging van wifi te appreciëren gaan we wederom even terug in de tijd om te kijken hoe de originele IEEE wifi standaard de beveiliging beschreef. Het zal een ietwat horror-achtige tocht worden waarin we gaan ontdekken dat enkele stevige hiaten ervoor gezorgd hebben dat illegale toegang tot bijna ieder wifi netwerk rond de eeuwwisseling binnen enkele minuten kon gebeuren. Lees verder en huiver mee.

Onbeveiligde managementframes

Veel keuzes die in de IEEE standaard werden gemaakt zijn vermoedelijk het gevolg dat men leentje buur is gaan spelen bij de reeds bestaande IEEE standaarden voor bedrade netwerken. Echter, bij bedrade netwerken had je niet de inherente onveilige omgeving van het draadloze aspect, waardoor op gebied van beveiliging hier weinig tot geen aandacht aan werd besteed.

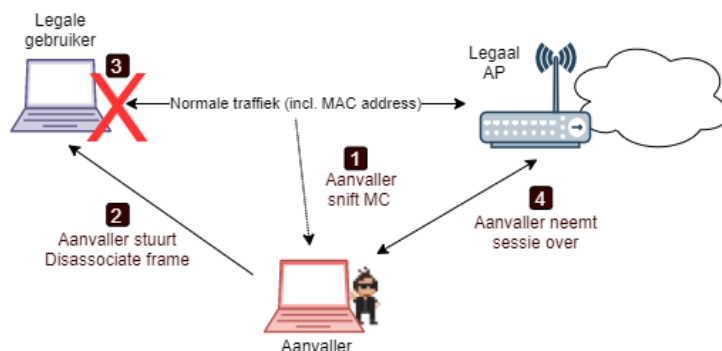
Management frames in wifi zijn frames die ervoor zorgen dat alle aanwezigen op het netwerk op een ordentelijke manier met elkaar kunnen communiceren. Deze frames zorgen bijvoorbeeld voor:

- Om een client verbinding te laten maken met een netwerk.
- Om SSIDs te broadcasten.
- Clients de opdracht te geven het netwerk te verlaten.
- Om te verbinden met een ander access point van hetzelfde netwerk (*handover*).

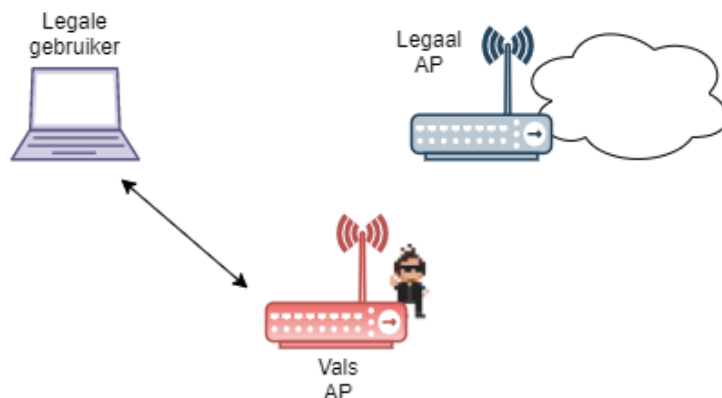
Net zoals bij bedrade netwerken, koos men bij de IEEE wifi standaard om deze managementframes **zonder enige vorm van beveiliging** te gebruiken (er werd geen *CIA* voorzien). Dit zorgde ervoor dat niet legale gebruikers zelfs management frames konden uitsturen op een netwerk, zonder dat de ontvangers ervan konden controleren of de bron wel een legaal access point of client was.

Dit resulteerde in onder andere volgende scenario's:

- Een hacker kan legale gebruikers DoS'n door constant zogenaamde *disassociation* naar hen te sturen. Dit frame, gebruikt door access points, geeft aan clients de opdracht dat ze het netwerk moeten verlaten. De hacker kan zo'n frame uitzenden en daarbij het "source" veld instellen op het MAC-adres van het access points. Dit spoofing kan ongecontroleerd, waardoor legale gebruikers dit frame altijd zullen aanvaarden én vervolgens uitvoeren: de gebruiker kan niet meer verbinden met het netwerk zolang de disassociation frames blijven verstuurd worden (**disassociation flooding**)
- **Identity spoofing** was ook eenvoudig daar de hacker eender welk veld in de frames kan aanpassen. Van zodra hij een legale gebruiker met voorgaande techniek van het netwerk heeft geschopt, kan hij vervolgens zichzelf voordoen als deze gebruiker. Hiervoor moet hij gewoon het MAC-adres spoofing van de legale gebruikers tijdens de communicatie met het access point.



- En *last but not least* laten de onbeveiligde management frames toe dat we eenvoudig een access point kunt nabootsen (**impersonation**). Vervolgens kunnen we een **man-in-the-middle aanval** uitvoeren daar een hacker zich kan plaatsen tussen de gebruiker en het internet en zo informatie kan ontfutselen.





De linux tool *AirSnarf* laat toe om fake hotspots (publiek wifi netwerk) op te zetten. Hierbij maakt het gebruik van de onbeveiligde management frames. Een scenario in België dat gegarandeerd success heeft (vanuit het standpunt van de hacker) is een fake Telenet Wifree hotspot op te zetten. Hierbij zal je eerst de login pagina van Telenet Wifree kopiëren en via een lokale webserver aanbieden aan de gebruikers die op jouw access point met de naam “Telenet Wi-Free” verbinden. Je kan nu van iedere gebruiker de gebruikersnaam en paswoord stelen telkens deze die informatie op jouw fake pagina invoert.

De 802.11 standaard qua beveiliging

De originele “ANSI/IEEE Std. 802.11” die de wifi specificaties beschrijft werd geschreven in 1999. Het had als doel “*to develop a medium access control (MAC) and physical layer (PHY) specification for wireless connectivity for fixed, portable, and moving station within a local area.*” Hoofdstuk 8 van deze standaard had als titel “Authentication and privacy” en was maar 10 miezerige pagina’s lang, in vergelijking de totale grootte van het document (528 pagina’s) was dit misschien wel een voorbode hoe weinig aandacht er aan beveiliging zou worden gegeven.



Vanaf nu zullen we geregeld het woord access point afkorten naar **AP**. In deze tekst kan een AP zowel een eenvoudig access point zijn als een complexe draadloze router of modem.

Voor we dat kleine hoofdstuk gaan openbreken -en ontdekken hoe WEP in de eerste generatie wifi-apparaten een navenante beveiliging aanbod- zullen we eerst bekijken hoe gebruikers met een draadloos netwerk effectief kunnen verbinden. Zoals reeds vermeld gebeurt dit gebruik makende van de onbeveiligde management frames.

Verbinden met een netwerk

Voor aanvallers actieve aanvallen kunnen starten op een netwerk dienen ze verbinding te maken met het netwerk. Dit proces bestaat uit 4 stappen die doorlopen moeten worden voor legale en illegale gebruiker effectief gebruik kunnen maken van het netwerk en *higher-layer* datatraffiek kunnen verwerken.

1. **Scannen:** Passief of actief zoeken naar de netwerken in de buurt.
2. **Verbinden (*joining*):** Kiezen met welk netwerk zal verbonden worden.
3. **Authenticatie:** Bewijzen dat de gebruiker toegang heeft tot het netwerk.

4. **Associatie (*association*)**: Bijhouden met welk AP de gebruiker verbonden is van het netwerk. Pas vanaf deze stap kan het netwerk gebruikt en misbruikt worden.

Scannen

In deze stap zal de client ontdekken welke netwerk er in de omgeving zijn. Oftewel zoekt de client specifiek naar een netwerk, oftewel wil hij gewoon een olijsting van alle aanwezige netwerken. Ieder draadloze netwerk heeft een netwerdnaam, de **SSID** (*service set identifier*) die bestaat uit een 32 byte ASCII character string, en zal deze op geregeld tijdstip broadcasten voor iedereen die hier nood aan heeft (we merkten al eerder op dat dit broadcasten kan uitgezet worden, maar dat dat een vals gevoel van veiligheid zal geven). Naast het SSID zal ieder netwerk ook meerdere fysische parameters uitsenden die de client in de volgende stap zal nodig hebben.

Er zijn meerdere kanalen beschikbaar in het spectrum waar binnen een netwerk mag werken. De client zal bij het scannen daarom steeds enkele milliseconden op een bepaald kanaal luisteren om potentiële SSID broadcasts op te vangen, om dan naar het volgende kanaal hetzelfde te doen.

Verbinden

Wanneer de gebruiker gekozen heeft met welk SSID er moet verbonden worden zal in deze stap de wifi-netwerkaart ingesteld worden op de juiste fysische parameters (frequentie kanaal, snelheid, etc.) zodat als het ware AP en client synchroon lopen (en dus op de juiste moment iets tegen elkaar kunnen zeggen zonder elkaars signalen te storen). In de scan-stap keken we als het waren goed rond, in deze stap richten we onze blik nu op een bepaald netwerk.

Authenticatie



Deze en de voorgaande stappen gebeurt volledig automatisch indien de gebruiker eerder heeft geopteerd om automatisch met een netwerk te verbinden.

In een bedraad netwerk zit authenticatie impliciet vervat in het fysisch aspect: wanneer je de kabel in je laptop kan steken dan is de kans groot dat je een legale gebruiker bent in het gebouw en dus is er geen extra authenticatie nodig (kort door de bocht gezien, weliswaar). Dat is niet zo met draadloze netwerken die voorbij de grenzen van het gebouw hun signaal uitsenden. Er is daarom een extra stap nodig voor we het netwerk kunnen betreden.

In de 802.11 standaard staan 2 mogelijke authenticatie-methoden beschreven (die we verderop zullen toelichten):

- **Open-system authentication**
- **Shared-key authentication**

Origineel was deze authenticatie een enkelrichtingsstraat. Enkel de client dient zich te authenticeren. Hierdoor bestaat dus de kans dat de gebruiker verbind met een rogue of fake AP.

Nast voorgaande 2 methoden is er nog een derde die niet in de standaard staat beschreven maar die wel door veel fabrikanten werd aangeboden in hun wifi-apparaten:

- **MAC Address Authentication gebruik makend van een MAC-ACL:** hierbij kan de beheerder van een AP een *access control list* (ACL) aanleggen waarin alle MAC-adressen staan van toegelaten apparaten.

Open-system authentication

Deze vorm van authenticatie werd eerst gezien als de “er is geen authenticatie nodig op dit netwerk”-versie van authenticatie. Echter, de problemen met WEP op gebied van beveiliging hebben er voor gezorgd dat sinds de WEP-opvolger WPA, deze vorm de veiligere van de twee modes is.

In netwerken met deze modus om te authenticeren worden er exact 2 frames tussen client en AP uitgewisseld:

1. Eerst vraagt de de client aan het AP toegang.
2. Vervolgens stuurt het AP naar de client een “Welkom” frame.

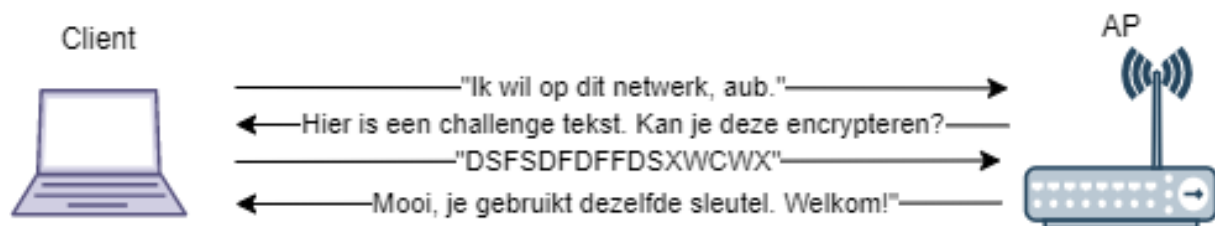
Meer gebeurt er niet in deze modus.

Als er in dit soort netwerk geen encryptie wordt toegepast dan kan dus eender welk apparaat in de omgeving verbinding maken met dit netwerk. Als er wél WEP encryptie wordt gebruikt dan zal het bezit van de WEP-sleutel als een soort toegangscontrole werken: je kan namelijk wel authenticeren (daar het AP iedereen toelaat in deze authenticatie-fase) maar vervolgens kan je geen data lezen en versturen tenzij je een geldige WEP-sleutel hebt.

Shared-key authentication

In deze modus moet je bewijzen dat je in het bezit bent van een geldige WEP-sleutel voor dit netwerk. Het AP zal daarom een **challenge-response** authenticatie opstarten bestaande uit volgende sequentie van frames:

1. Het AP maakt een random string aan, de *challenge*, en stuurt deze in plaintext naar de client.
2. De client encrypteert deze string met z’n WEP-sleutel en stuurt dit terug naar het AP (de *response*).
3. Het AP zal nu de response decrypteren met z’n eigen WEP-sleutel en het resultaat vergelijken met de challenge-string. Als beide gelijk zijn weet het AP dat de client een geldige WEP-sleutel heeft en dus toegelaten mag worden op het netwerk.



Merk op dat stap 1 en 2 ervoor zorgen dat aanvallers die deze *handshake* sniffen 2 interessante frames zien passeren. Eerst zien ze een plaintext, ogenblikkelijk gevolgd door de bijhorende ciphertext ervan (indien ze een gebruiker sniffen met een geldige WEP-sleutel). Dit zal interessante informatie blijken verderop in dit horror-verhaal waarin we zullen tonen waarom WEP niet zo veilig bleek te zijn als gehoopt.

Associatie

Na een succesvolle authenticatie krijgt de client een *association ID* toegewezen. Dit ID gebruikt het netwerk om te weten waar in het netwerk de client zich bevind. Veel draadloze netwerken bestaan namelijk uit meerdere AP's en via dit id weet het netwerk met welk AP de client momenteel verbonden is en zal alle data voor de client dan naar dat AP sturen.

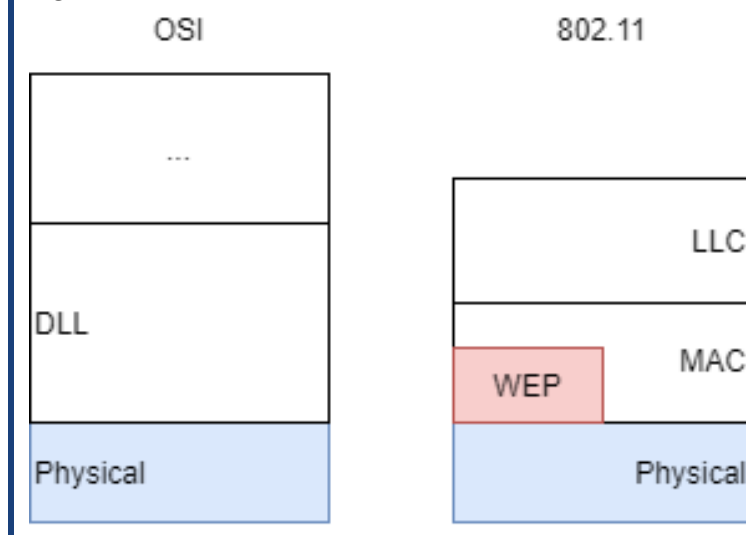
Vanaf dit punt kan de gebruiker dus de bronnen van het netwerk beginnen gebruiken en wordt het tijd om deze communicatie te beveiligen (tenzij het om een publieke hotspot gaat waar iedereen alles van elkaar kan zien).

WEP

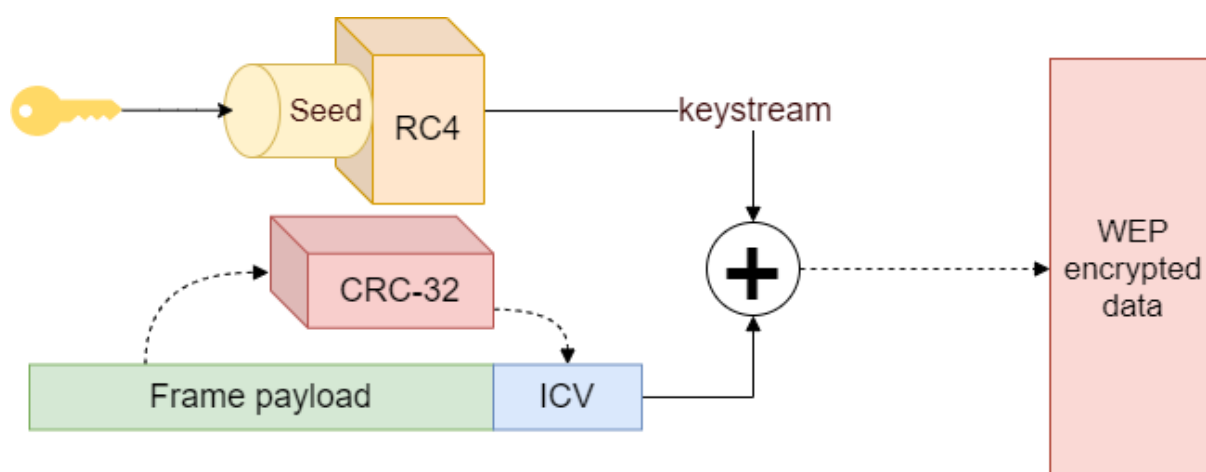
Om potentiële aanvallers ervan te weerhouden dat ze traffic kunnen sniffen (of zelf op het netwerk zetten) voorziet de 802.11 vanaf de associatie de optie om encryptie te voorzien. Dit gebeurt aan de hand van **WEP**, wat staat voor *wired equivalent privacy*. Een naam die veel beloofde maar niet zo goed was: de idee was dat WEP even veilig zou zijn als een bedraad netwerk.



WEP is optioneel en bevindt zich vlak voor frames naar de fysische laag (*PHY*) worden gestuurd die de frames “in de lucht” zal sturen. Het IEEE werkt met lagen die ongeveer overeen komen met de OSI-lagen maar met iets andere namen. In volgende figuur zie je waar WEP, optioneel, zich bevindt ten opzichte van de onderliggen en bovenliggende lagen.

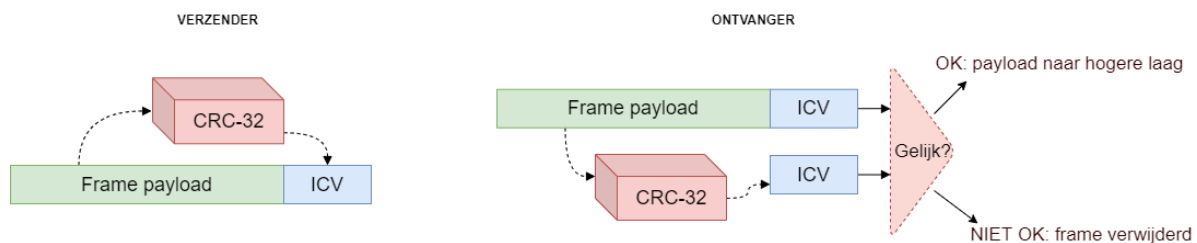


Hoe werkt WEP?



Figuur 0.2: WEP in z'n geheel

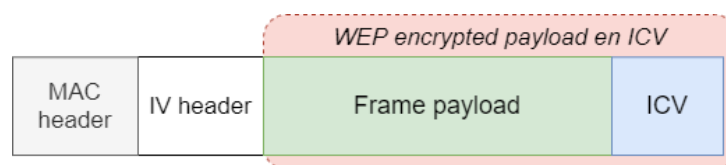
Het hart van WEP is het RC4-algoritme dat we reeds zagen in het crypto-hoofdstuk. De WEP-sleutel zal dienst doen als de seed voor de keystream generatie. Deze keystream zal op zijn beurt ge-XOR'd worden met het te encrypteren frame.



Figuur 0.3: Integrity check

Confidentiality en integriteit worden tegelijkertijd afgehandeld in WEP. Vlak voor dat het frame via de XOR-operatie wordt geëncrypteerd zal het frame eerst door een *integrity check algoritme* gestuurd worden. Deze integrity check gebeurt met behulp van CRC-32, een oude getrouwe op dit gebied. CRC-32 zal een hash genereren die de ontvanger bij ontvangst kan gebruiken om te zien of het frame werd aangepast na verzenden (bewust door een aanvaller, of door bijvoorbeeld ruis in het netwerk). Deze hash, de **Integrity Check VALUE (ICV)**, zal mee worden geëncrypteerd door RC4 voor hij verstuurd wordt. Op deze manier kunnen ordinaire aanvallers het frame niet aanpassen zonder dat ze daarmee ook de ICV ongeldig maken.

Finaal verkrijgen we dus volgende WEP-frame dat kan verstuurd worden:



Figuur 0.4: WEP frame layout



In de originele standaard zat de mogelijkheid om tot 4 WEP-sleutels in een netwerk te gebruiken. Via de *keyid* kon een client of AP dan aangeven met welk van de geïnstalleerde sleutels een frame werd geëncrypteerd.

Sleutellengte en de IV

Origineel ondersteunde WEP enkel 40-bit WEP sleutels. Ondertussen is dat opgetrokken maar toen de standaard werd geschreven besepte men al dat er sowieso een probleem met de sleutel zou zijn als die zo zou gebruikt worden: ieder frame dat met dezelfde sleutel wordt geëncrypteerd zal dezelfde keystream hebben gebruikt. Dat was natuurlijk geen optie. Om die reden werd gekozen om te werken met een **initialisatie vector (IV)** van 24-bit. Deze IV werd mee als seed aan RC4 gegeven. Door ieder frame een andere IV te kiezen zorgden men er zo voor dat ieder frame een andere keystream gebruikte

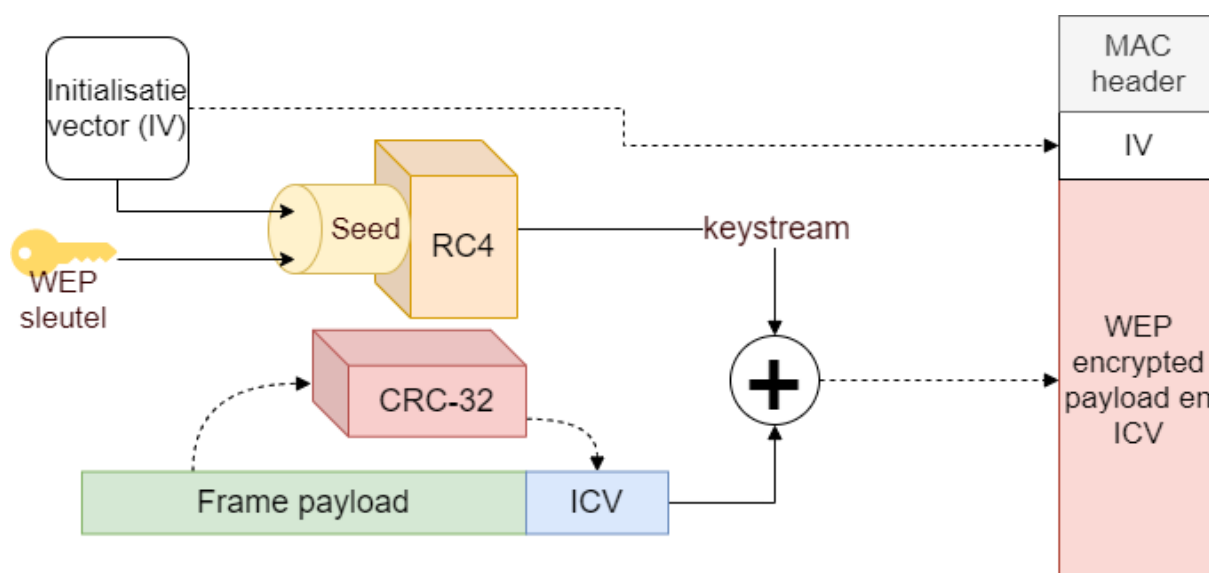
(WEP-sleutel+IV werd de nieuwe seed per frame). In de originele standaard werd echter niet beschreven hoe deze IV moest veranderen wat nefaste gevolgen zal hebben verderop.

Omdat ook de ontvanger dezelfde keystream moet kunnen genereren tijdens decryptie is het natuurlijk belangrijk dat de IV ook bij de ontvanger gekend is. De enige manier om dit op te lossen is door de IV mee in de header van het frame te plaatsen en door te sturen. Uiteraard moet deze IV als plaintext door het leven gaan.

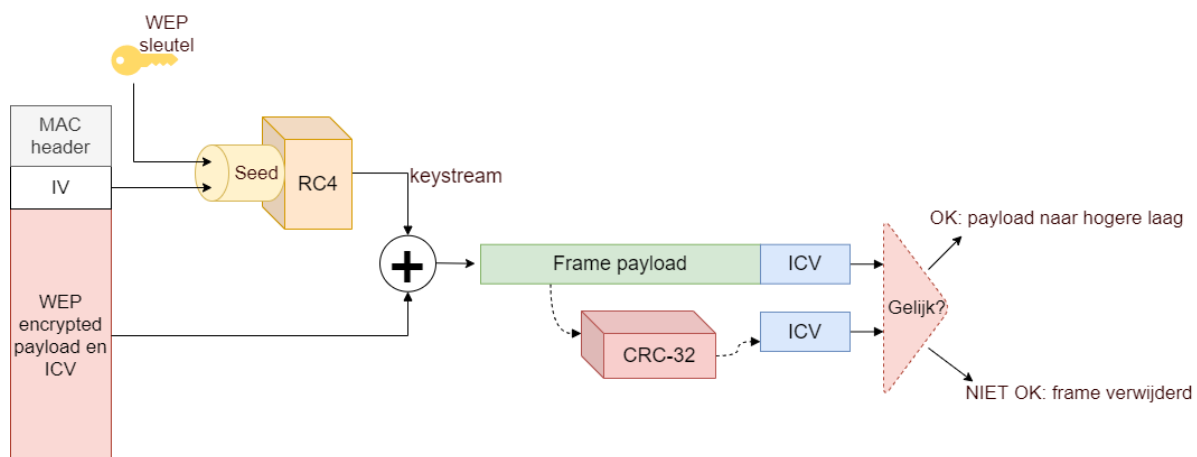


Dit concept van een sleutel verlengen met een arbitrair getal heet *salting* en zullen we in het hoofdstuk omtrent paswoorden en authenticatie verderop in de cursus nog zien terugkomen.

Finaal krijgen we dus de volgende werking, encryptie en decryptie, als volgt:



Figuur 0.5: WEP encryptie



Figuur 0.6: WEP decryptie

Hoe WEP faalde

Het leek een verbonden vat: hoe populairde wifi werd over de hele wereld, hoe meer papers er verschenen die fouten identificeerden in WEP. Al vrij snel werd duidelijk dat WEP hoegenaamd géén CIA kon aanbieden. De meeste fouten die werden gevonden kunnen gegroepeerd worden in volgende 4 zaken:

1. Het **RC4 algoritme** is helemaal niet gemaakt voor een datagramnetwerk zoals wifi.
2. De manier waarop de **IV** in de standaard is beschreven is ontoerijkend en verhoogd de kans op foute implementaties door fabrikanten.
3. **CRC-32** kan omzeilt worden en kan dus geen integriteit van frames garanderen.
4. Er is **geen sleutel-management systeem**.

Er is echter nog een vijfde fout die de voorgaande 4 als het ware nog versterkt:

5. Er is geen **replay protection**.

Hierdoor heeft de aanvaller dus vrij spel en kan hij draadloze netwerk als een soort experimenteertuin gebruiken en duizenden pakketjes te pas en te pas onpas heruitzenden. We zullen nu de eerste 4 grote problemen beschrijven. De *replay protection* behandelen we niet apart maar zullen we geregeld bij de andere problemen zien opduiken.

Probleem 1: RC4

De meeste problemen met WEP komen van een verkeerd gebruik van het RC4 algoritme. RC4 wordt in erg veel moderne beveiligingsapparaten toegepast omdat het een sterk én efficient algoritme is (het

verbruikt weinig energie omdat er geen dure vermenigvuldigingsoperaties in voorkomen). Echter, stream ciphers in het algemeen, RC4 specifiek, zijn eigenlijk geen goede keuze voor datagram netwerken waarin transmissies onbetrouwbaar zijn.

In een datagram netwerk worden pakketjes vaak opnieuw verstuurd wanneer er een fout optrad en de ontvanger voor een *retransmission* vraagt (dit gebeurt voor ongeveer 20% van de verstuurde data). Dit is volledig normaal gedrag in zowel bedrade als draadloze netwerken, echter voor een stream cipher is dit nefast. Specifiek 2 eigenschappen van stream ciphers (en dus ook RC4) zorgen voor stevige gebreken in het WEP-protocol inzake confidentiality:

1. RC4 heeft **geen random access mogelijkheden**.
2. RC4 staat **geen sleutel hergebruik** toe.

Laten we die 2 eigenschappen eens bekijken en welke cascade van problemen ze met zich meebrengen.

RC4 heeft geen random access mogelijkheden Deze eigenschap is niet zo zeer een probleem vanuit beveiligingsperspectief, maar wel vanuit performantieperspectief. RC4 had eigenlijk nooit gekozen mogen worden door het IEEE om in WEP gebruikt te worden. Het verlies van één bit van de datastroom zal ervoor zorgen dat alle bits erna met RC4 ook verloren zijn, daar we met een stream cipher werken waarbij de synchronisatie van de stroom bits tussen verzender (encryptie) en ontvanger (decryptie) gelijk moet blijven. Bij het minste dataverlies moeten beide zijden hun *RC4-motortje* resetten en opnieuw beginnen.

AES bijvoorbeeld heeft wél die random access mogelijkheid: hierdoor kan steeds herbegonnen worden aan het punt van dataverlies en niet helemaal opnieuw, wat natuurlijk veel efficiënter is (daar we werken in een datagram omgeving waar bitverlies bijna continue voorkomt).

RC4 staat geen sleutel hergebruik toe Stream ciphers hebben een tweede erg belangrijke eigenschappen: **het is uiterst onveilig om een zelfde sleutel twee keer te gebruiken!**

Stel dat je volgende 2 plaintext byte sequenties hebt: $p_1, p_2, p_3, \dots$ en $q_1, q_2, q_3, \dots$. Beide worden met dezelfde keystream $k_1, k_2, k_3, \dots$ geëncrypteerd. Dit geeft ons vervolgens volgende 2 ciphertext sequenties:

$$p_1 \oplus k_1, p_2 \oplus k_2, p_3 \oplus k_3$$

$$q_1 \oplus k_1, q_2 \oplus k_2, q_3 \oplus k_3$$

Als we nu veronderstellen dat een aanvaller deze 2 ciphertexts capteert, wat dan volgt is een grove schending van de confidentiality die RC4 zou moeten garanderen:

$$(p_i \oplus k_i) \oplus (q_i \oplus k_i) = p_i \oplus q_i$$

Of in andere woorden, wanneer we de beide ciphertexts met elkaar XOR'n krijgen we een sequentie die **niet afhankelijk is van de gebruikte sleutel!** Een stevige hoeveelheid informatie over beide plaintext wordt zo onthult. Als 1 van beide plaintexts gekend is dan volstaat een eenvoudige XOR-operatie om ook de andere plaintext te kunnen zonder dat hierbij de gebruikte sleutel moet gekend zijn. Deze fout zullen we verderop misbruiken.

Kortom, stream ciphers zijn niet veilig in een datagram omgeving indien er geen vorm van sleutelmanagement bestaat die de sleutels kan vervangen voor ze herbruikt worden. WEP probeert dit gebrek aan sleutelmanagement te omzeilen door met een IV te werken zodat er geen collisions zoals eerder beschreven kunnen optreden... maar ook dat zal een resem problemen met zich meebrengen.

Probleem 2: IV

Het IEEE had dus weet van voorgaand probleem met RC4 en introduceerde daarom het IV. Vanuit cryptografisch standpunt is dit een solide oplossing. Echter, door het gebrek aan replay protection krijgen we helaas een hoop fouten met de IV.

De IV veroorzaakt weak keys In een paper van 2001 door Scott Fluhrer, Itsik Mantin en Adi Shamir werd aangetoond dat het key scheduling algoritme (KSA) van RC4 een hiaat bevat:

1. Wanneer een deel van de gebruikte RC4 keystream gekend is dan kan een groep *RC4 weak keys* gevonden worden.
2. Wanneer deze weak keys gebruikt worden om een keystream te genereren dan zal er informatie van de gebruikte WEP sleutel in deze keystream gelekt worden.

Of anders gezegd: sommige IV's zorgen voor een sleutel-lekkage naar de keystream, wat natuurlijk nefast is voor eender welk cryptografisch cipher.

Een gevolg van die weak keys is dat, indien de eerste 2 bytes van genoeg keystreams (ongeveer 60) geweten is, men de gebruikte WEP sleutel kan achterhalen door middel van een FMS aanval (de afkorting staat voor de eerste letters van de 3 onderzoekers uit de paper).

De FMS aanval werkt indien:

1. We ongeveer 60 keystreams kunnen capteren waarvan geweten is dat ze *weak* zijn.
2. We de eerste 2 bytes van de plaintext van de bijhorende frames kennen die met deze weak keystreams zijn geencrypteerd.

Dat tweede is geen probleem, met dank aan de netwerkspecificaties: de payload van een met WEP geëncrypteerd pakket bevat de LLC header (de header van de logical link layer). Volgens de standaard (RFC 2684) moeten "IP datagram pakketten altijd zichzelf in de header identificeren via de SNAP header"

En laten de eerste 2 bytes van die header toch wel niet altijd starten met 0xAA. Kortom, quasi alle frames die over een WEP-netwerk vliegen zullen altijd met de hexadecimale waarde AA starten. Nu volstaat het om de bijhorende keystream te achterhalen aangezien we de plaintext kennen, kunnen we ook de eerste 2 bytes van de keystream kennen daar we weten dat:

$$c_i = k_i \oplus p_i$$

(waarbij p_i AA is)

En dus:

$$c_i = k_i \oplus p_i \Leftrightarrow k_i = c_i \oplus p_i$$

We hebben zo ook deel 1 van de FMS in onze handen en kunnen nu het algoritme de gebruikte WEP-sleutel laten berekenen (de manier waarop dat gebeurt zou ons te ver brengen)/



De FMS aanval is geïmplementeerd in volgende 2 erg populaire Linux tools: Aircrack-ng & WEPCrack. Beide kunnen dus gebruikt worden om de WEP-sleutel van een draadloos netwerk te achterhalen.

IV collisions treden op Op zich is de FMS aanval al dramatisch, maar helaas stopt het hier niet. Doordat de IV maar 24 bit groot is treden er veel sneller collisions op dan intuïtief wordt verwacht. Van zodra 2 pakketjes met dezelfde IV zijn verstuurd treedt er een collision op, en die zijn erg interessant voor aanvallers. Pakketjes met dezelfde IV zijn pakketjes waarvan de payload met dezelfde keystream werd geëncrypteerd.

Een 24-bit IV kan 2^{24} oftewel 16 777 216 mogelijke waarden hebben. Een kleine berekening toont hoe snel collisions optreden:

Gegeven: een eerste generatie AP die aan een miezerige 11 Mbps werkt en continue 1.500-byte pakketjes in de lucht stuurt:

- $\frac{11 \text{ Mbps}}{(1500 \text{ bytes/pakket}) * 8 \text{ bits/byte}} = 916.67 \text{ pakketjes/seconde}$
- $\frac{16.777.216 \text{ IV's}}{916.67 \text{ pakketjes/seconde}} \approx 18302 \text{ seconden}$

Dat wil dus zeggen dat na ongeveer 5uur alle IV's opgebruikt zijn en er dan ten laatste collisions optreden.

Deze fout kunnen aanvallers op 2 manieren misbruiken: met een passieve of met een actieve aanval.

Passieve IV aanval

Een aanvaller kan passief meeluisteren (*eavesdropping*) en stilletjes alle traffic onderscheppen tot er een IV collision optreedt. Door twee pakketjes met eenzelfde IV te XOR'n verkrijgt de aanvaller een

pakket dat bestaat uit de XOR van beide plaintext'en van de gecapteerde pakketten. Als dus één van beide plaintexten gekend is, is de inhoud van het andere pakket ook gekend.

IP traffic is vaak erg voorspelbaar en bevat aardig wat redundantie (om fouten op te vangen). Hierdoor wordt het makkelijker voor een aanvaller om via cryptanalysis te achterhalen wat de inhoud, of een deel, van het pakket bevat. Een voorbeeld hiervan toonden we bij de FMS aanval waarbij steeds de LLC header gekend was van de meeste pakketten.

Omdat collisions redelijk snel optreden (vergeet niet dat het voorbeeld hierboven maar sprak over één client en één AP. Als er dus meerdere clients actief zijn in een netwerk treden collisions véél sneller op) is het voor een aanvaller dus maar een kwestie van lang genoeg te sniffen om zo een grote hoeveelheid pakketten met gelijke IV's op te vangen, waardoor de cryptanalysis ongelooflijk vereenvoudigd wordt.



In al deze voorbeelden gaan we er vanuit dat de gebruiker geen encryptie toepast op de hogere lagen waar z'n data vandaan komt. Uiteraard wordt cryptanalyse een pak moeilijker als de payload van gecapteerde pakketten geëncrypteerd blijkt te zijn.

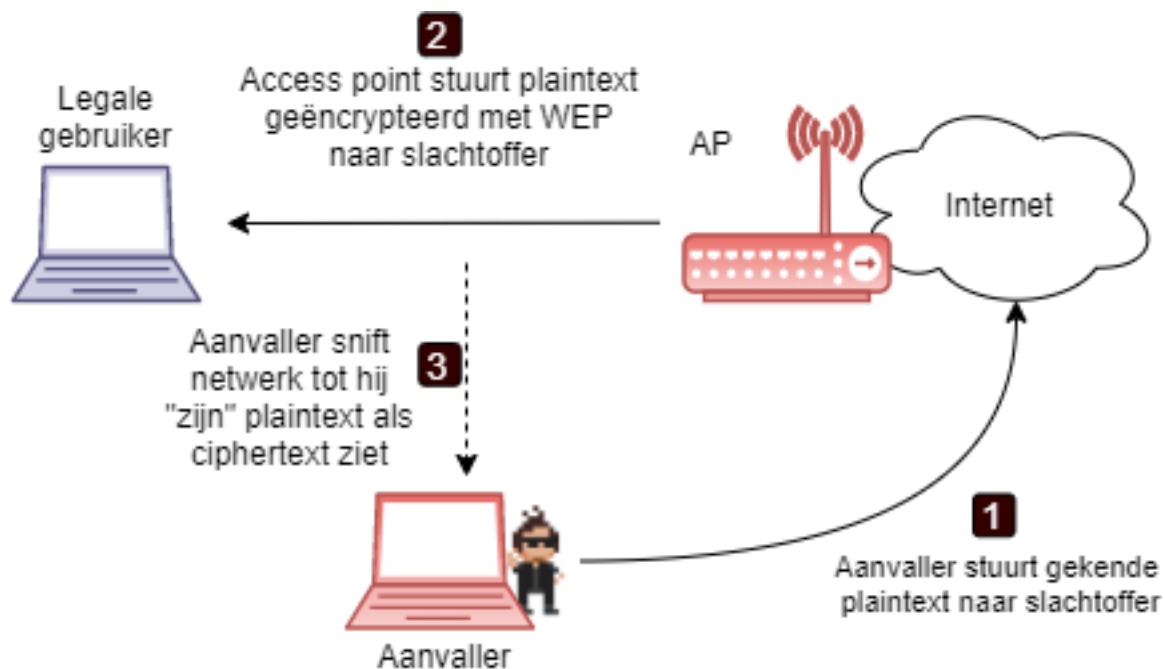
Actieve IV aanval

De passieve aanval heeft als nadeel dat we als aanvaller:

1. moeten wachten op collisions, en dus bijgevolg op traffic over het netwerk.
2. we enkel door weloverwogen gokken (cryptanalyse) kunnen proberen te weten te komen wat de originele plaintext juist is.

Beide problemen kunnen we als aanvaller echter te niet doen door een actieve rol te gaan spelen. Doordat een AP braaf alle traffic encrypteert dat het van het bedrade netwerk krijgt om naar een client te sturen, is het voor een aanvaller een kwestie van "gekende" plaintext van buitenuit naar het slachtoffer te sturen. Als volgt:

1. Een gekende plaintext boodschap (bijvoorbeeld een emailbericht of ping) wordt naar het AP gestuurd (via het internet bijvoorbeeld), dat vervolgens door de aanvaller in het oog wordt gehouden. Noot: als de aanvaller enkel het draadloze netwerk ter beschikking heeft (en niet het internet) dan zal een bitflip-aanval moeten gebruikt worden, wat we verderop zullen uitleggen.
2. De aanvaller blijft sniffen tot het de ciphertext ziet passeren waarin (vermoedelijk) z'n gestuurde plaintext zit.
3. Vervolgens kan de aanvaller een keystream te pakken krijgen door z'n plaintext te XOR'en met de gecapteerde ciphertext: $c_i = k_i \oplus p_i \Leftrightarrow k_i = c_i \oplus p_i$.



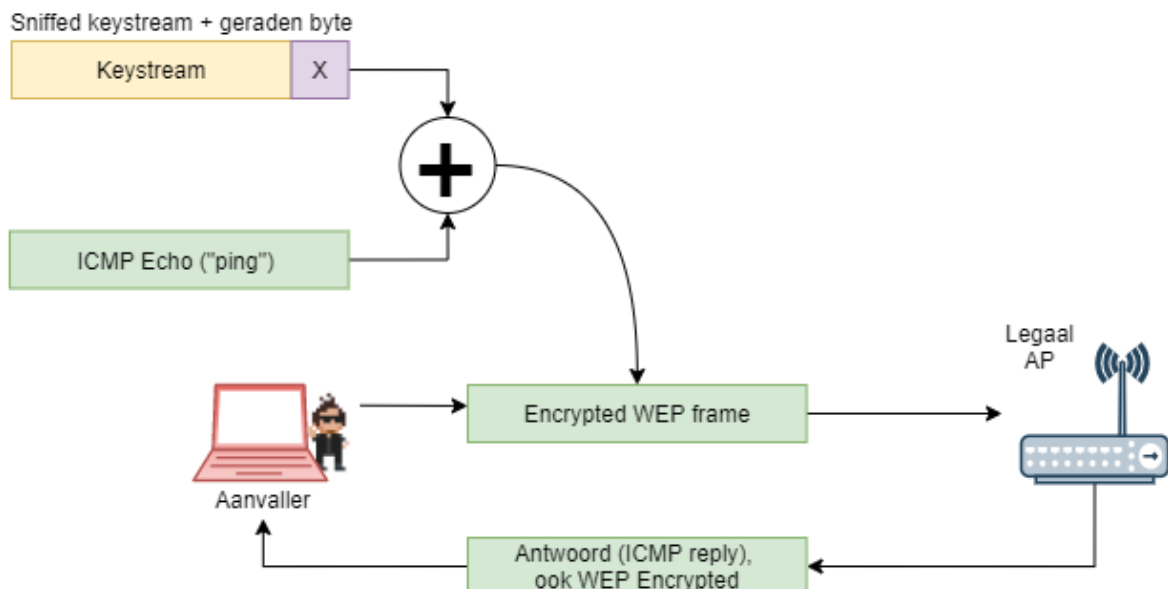
Keystreams groeien Wanneer een aanvaller met voorgaande IV collisions keystreams kan capteren kan hij in principe data op het netwerk beginnen plaatsen (aangezien het netwerk ervan uitgaat dat het gebruiken van geldige keystreams, wil zeggen dat de gebruiker geauthenticeerd is omdat hij de bijhorende WEP-sleutel heeft). De aanvaller kan nu plaintext XOR'n met deze gevonden keystream en op het netwerk zetten. Echter, hij is beperkt tot pakketten die even lang zijn als de keystream die gevangen werd. Het zou véél nuttiger zijn als de aanvaller als het ware een bibliotheekje heeft van geldige keystreams van allerlei lengtes.

Omdat er geen replay protection aanwezig is, kan de aanvaller heel eenvoudig z'n gecapteerde keystreams doen *groeien* en zo byte per byte een langere keystream genereren. Dit gaat als volgt te werk:

1. De aanvaller maakt een plaintext pakketje aan dat 1 byte langer is dan de keystream die hij al heeft. Het ping-commando (ICMP) kan je met de "-l" optie bijvoorbeeld een ping van eender welke bytesize laten genereren. Het voordeel van het ping-commando gebruiken is ook dat we exact weten wat voor response er kan verwacht worden.
2. De aanvaller plakt nu 1 byte achter de gecapteerde keystream. Hij kiest hierbij een willekeurige waarde en heeft dus 1 kans op 256 om de juiste te kiezen.
3. De aanvaller XOR'r deze keystream met het commando uit stap 1 en stuurt dit op het netwerk.
4. Indien de aanvaller in stap 2 de juiste byte gekozen heeft dan zal er een reactie op de ping volgen (daar het pakket werd gedecrypteerd door het AP en dan hoger in de OSI-stack door het netwerk werd gestuurd). Als de keystream fout is zal er geen reactie komen daar het AP het pakketje als

foutief heeft weggegooid en dus heeft genegeerd.

5. Als een verkeerde byte werd gekozen in stap 2 dan zal de gebruiker dit proces opnieuw starten en nu een andere byte-waarde kiezen. Hij zal dit blijven herhalen tot hij in stap 4 reactie krijgt en dus weet dat hij nu z'n keystream met succes heeft doen groeien met 1 byte.



We hebben bij deze aanval 1 belangrijk concept genegeert waardoor deze aanval op eerste zich niet mogelijk is: het aanpassen van een payload resulteert ook in een nieuwe CRC. Deze is echter niet gecrypteerd waardoor het niet duidelijk is hoe we dit kunnen omzeilen. Wacht nog even tot we aan de bitflip aanval komen en alles zal duidelijk worden (dat dit dus geen probleem is).

IV selectie De vierde fout met de Initialisatie Vectors is de manier waarop de selectie ervan moet gebeuren in de hardware. De 802.11 gaf enkel aan dat het IV *“geregeld moest geupdate”* worden. Dat is uiteraard te vaag en heeft ervoor gezorgd dat fabrikanten zelf moesten bepalen welke IV selectie strategie ze in hun hardware zouden implementeren. Hierdoor waren er 3 strategieën die hun weg in de verschillende apparaten vonden:

- **Vast IV:** Sommige fabrikanten hadden geen flauw benul wat het doel van de IV was vanuit cryptografisch standpunt en kozen daarom zelfs gewoon om alle pakketten steeds met het zelfde IV te versturen.
- **Willekeurig IV:** Andere fabrikanten verkozen het om hun hardware bij ieder pakketje een willekeurig IV te laten selecteren. Alhoewel dit uiteraard veel veiliger is dan een “vaste IV”-strategie, treden er toch veel sneller collisions op dan verwacht. Dit valt te verklaren door het zogenaamde

verjaardagenparadox (zie kader verder) dat verklaart waarom er reeds 50% kans op een collisions is na 4823 pakketjes. Dat wil dus zeggen dat al na enkele seconden er meestal collisions optreden.

- **Incrementele IV:** In deze strategie wordt een circulaire teller gebruikt waarbij het IV telkens met 1 wordt verhoogd wanneer een pakket moet worden verstuurd. Meestal begint deze teller op een vaste waarde. Dit zal er dan ook voor zorgen dat er een collisions optreedt van zodra een tweede apparaat zich op het netwerk begeeft en dus begint uit te zenden met het IV gelijk aan het IV van het allereerste pakketje dat het eerste apparaat gebruikte.

Kortom, een 24-bit *salt* is véél te klein in een omgeving met erg hoge data-rates zoals een draadloos netwerk. Dit probleem zou nog beperkt kunnen worden indien de originele WEP geregeld sleutels kon verversen, maar door het gebrek aan enig key management was dat dus uit den boze (want herinner je: de enige reden dat we IV's nodig hadden was omdat anders steeds dezelfde WEP-sleutel als seed werd gebruikt en dus alle keystreams gelijk zouden zijn. Door geregeld een andere sleutel te gebruiken zou onze kleine IV-lengte minder precair zijn, als we maar tijdig de sleutels verversen).



Volgende tekst uit Wikipedia legt de **verjaardagenparadox** uit: “De verjaardagenparadox is een paradox uit de kansrekening, die een resultaat toont dat tegen de verwachting ingaat. Het gaat om de vraag hoe groot de kans is dat in een groep willekeurig gekozen mensen er (minstens) twee dezelfde verjaardag hebben. Het blijkt dat, onder enkele lichte veronderstellingen, deze kans al meer dan 50% is voor een groep van maar 23 mensen. Bij 57 mensen is de kans zelfs meer dan 99%.”

Beeld je nu in dat in plaats van mensen, je honderden pakketten hebt, niet met een verjaardag maar met een eigen IV: de kans op collisions, ook al is de IV 24 bit, wordt dus 50% bij reeds een 5000 tal pakketten.

Probleem 3: CRC

WEP gebruikt een *integrity checksum field* om te voorkomen dat een pakket wordt aangepast tijdens transmissie, namelijk een CRC-32 checksum. Dit was niet zo'n wijze keuze: CRCs zijn in het algemeen vooral bedoeld om random transmissiefouten te detecteren, niet bewuste aanpassingen. Daarnaast heeft de CRC-32 ook nefaste gevolgen in combinatie met RC4 waardoor bitflipsaanvallen mogelijk zijn.

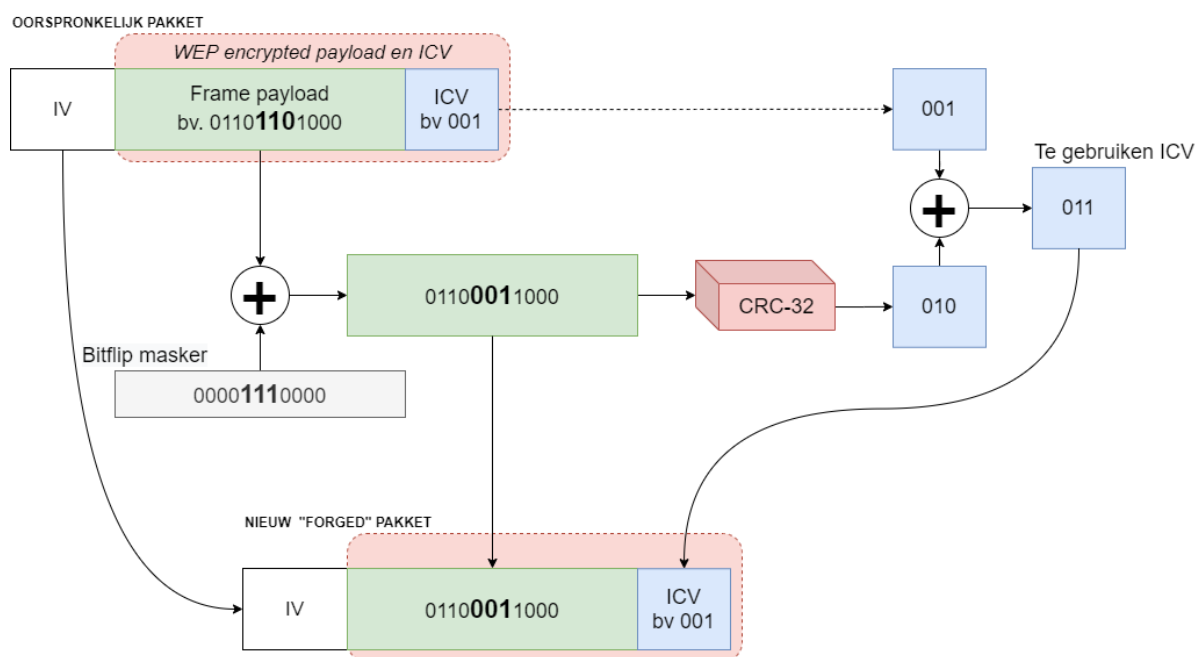
CRC-32 is een zogenaamde lineaire functie. Zonder in detail hierover in te gaan volstaat het te begrijpen dat als gevolg hiervan het volgende hebben: De CRC van 1 boodschap geXOR'd met de CRC van een andere boodschap is hetzelfde als de CRC nemen nadat beide boodschappen samen werden geXOR'd. Of beter gezegd:

$$CRC(\text{boodschap}_1) \oplus CRC(\text{message}_2) = CRC(\text{message}_1 \oplus \text{message}_2)$$

Door deze eigenschap kunnen we gecontroleerd aanpassingen aan pakketten doen, zonder daarmee de CRC te breken. Welgekome, bitflip aanval.

Bitflip aanval Om een bitflip aanval te doen heeft de aanvaller enkel 1 geldig WEP frame nodig. Hij hoeft zelfs niet te weten wat de inhoud ervan is, zolang het maar een geldig frame is.

Vervolgens maakt de aanvaller een bitflip masker: dit bestaat uit een reeks 0's, met 1 of 2 bits op 1. Dit zijn de bits die geflipt zullen worden in de volgende stap: de XOR nemen het bitflip masker met het payload gedeelte van het oorspronkelijke pakket. Welke bits geflipt worden doet er niet toe, integendeel: we willen net een geldig WEP-frame maken mét een foute data, zoals we zo meteen zullen zien. Deze nieuwe payload willen we nu in een geldig frame plaatsen, en dus hebben we een geldige ICV nodig. We doen dit door de ICV van de nieuwe payload te berekenen en deze te XOR'n met de, geëncrypteerde originele ICV. Het resultaat is een geldig, geëncrypteerde ICV voor de nieuwe payload. Bijgevolg hebben we een geldig frame kunnen maken dat door access points op het netwerk aanvaard zullen worden.

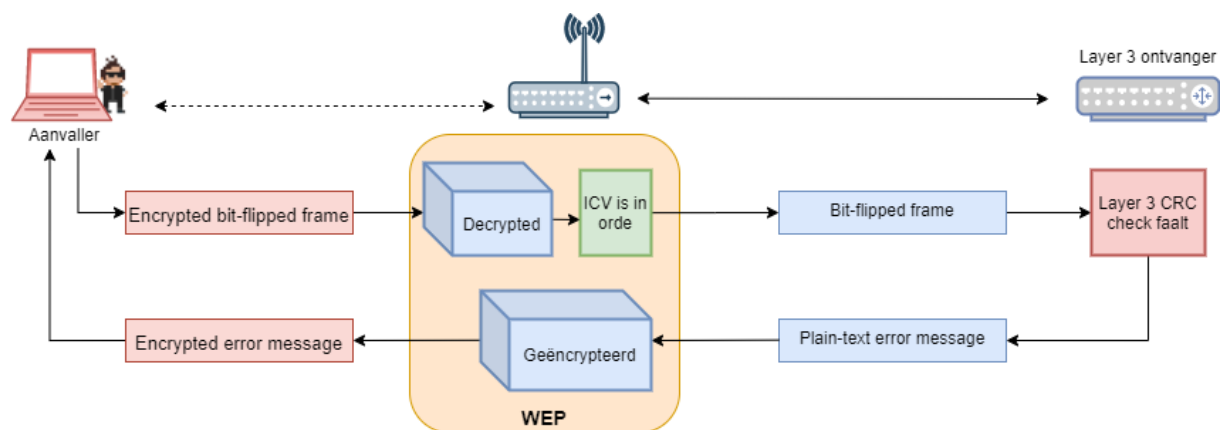


The attacker can now send a forged frame that is considered genuine by the receiver. The receiver (AP) dutifully decapsulates the bit flipped data and the LLC discovers that the data is 'gibberish'. Yet, since the ICV was valid, the receiver simply thinks some higher layer CRC error has occurred and thus sends an encrypted error-message to the attacker.

Wanneer een AP dergelijk pakket krijgt zal het dit pakket braaf naar de volgende laag sturen, ook al bevat de inhoud *rommel*, dankzij de geldige ICV. Op de volgende laag komt nu een pakket aan dat duidelijk stuk is, en deze laag zal bijgevolg een foutboodschap genereren en terugsturen. De

aanvaller krijgt deze boodschap in een WEP-frame aan. En alhoewel dit pakket geencrypteerd is, weet de aanvaller de inhoud van dit pakket (daar hij heeft opgezocht wat de foutboodschap zal bevatten op de volgende laag volgens de standaard van die laag) en kan dus een geldige keystream te pakken krijgen:

$$c = k \oplus p \Leftrightarrow k = c \oplus p$$



Dankzij de bitflip aanval kan de aanvaller dus zonder problemen keystreams groeien zoals eerder uitgelegd.

Probleem 4: Sleutelmanagement

Het moge duidelijk zijn: constant dezelfde WEP-sleutel gebruiken, op meerdere apparaten, gedurende meerdere dagen, is vragen om problemen. Werknemers die ontslagen worden kunnen een potentiële sleutel-lekkage veroorzaken met alle gevolgen van dien. Administrators moeten manueel nieuwe sleutels in voeren bij de werkgevers, etc. Kortom, 2 belangrijke oorzaken zorgen voor een nog lagere beveiligingsgraad van WEP dan er al was ten gevolge van de voorbije 3 problemen (IV, RC4, CRC):

1. Er is **geen geautomatiseerd sleutel verversmechanisme**: idealiter worden de sleutels binnen 1 sessie vervangen voor dat de pool van mogelijke IV's is opgebraakt.
2. Er is **geen gecentraliseerd sleutelmanagementsysteem**.

Hoe WEP oplossen?



Bart Preneel van de KUL/Cosic, één van België's meest vooraanstaande crypto-experts, zei ooit over WEP dat het het perfecte schoolvoorbeeld is van wat er allemaal kan fout lopen wanneer je beslist om zelf een nieuw crypto-algoritme te ontwikkelen.

Rond 2001, nog geen 2 jaar nadat de eerste wifi-standaard de wereld “het wonder van draadloze netwerken” bracht, werd duidelijk dat er dringend een oplossing moest verschijnen voor de vele veiligheidsproblemen. Wifi was alomtegenwoordig, zowel bij particulieren als in bedrijven, en dus moest een oplossing gezocht worden voor al die duizenden bestaande apparaten reeds in huizen en gebouwen. Het IEEE kon moeilijk een nieuwe standaard uitschrijven die alle bestaande gebruikers in de kou zette. Er werd daarom besloten om 2 pistes uit te werken:

1. WPA1: een (tijdelijke) oplossing voor bestaande apparaten, rekening houdend met enkele duidelijke beperkingen.
2. WPA2: de “ultieme oplossing” die een volledig nieuwe suite aan veiligheidsprotocollen zou bevatten voor toekomstige Wifi-producten, maar die niet compatibel zou zijn met bestaande apparatuur.



WPA staat voor Wi-Fi protected standard.

De beide oplossingen zouden ook de IEEE 802.1X standaard omarmen. Deze standaard zou sleutelmanagement en gebruikersauthenticatie voorzien in combinatie met de nieuwe encryptie en integriteitsoplossingen van WPA1 of WPA2.

Omdat sleutelmanagement én de bijhorende 802.1X infrastructuur redelijk overkill kon zijn voor huistuin- en keukegebruikers van wifi, besloot het IEEE om 2 versies van zowel WPA1 en WPA2 te maken:

- **Personal:** de thuisversie voor gewone gebruikers waarbij met een gedeelde sleutel of passphrase wordt gewerkt (een zogenaamde **PSK** oftewel *pre-shared key*). Hierbij wordt géén 802.1X gebruikt.
- **Enterprise:** de versie voor (grote) bedrijven waar sleutelmanagement belangrijk is en dus 802.1X wordt gebruikt

We vatten hier alvast de belangrijkste verschillen tussen WEP, WPA1 en WPA2 samen.

Standaard	Encryptie	Integrity	Authenticatie & sleutelmanagement
WEP	RC4	CRC-32	Geen
WPA1-Personal	TKIP	Michael	pre-shared key
WPA1-Enterprise	TKIP	Michael	802.1X
WPA2-Personal	CCMP	CBC-MAC	pre-shared key
WPA2-Enterprise	CCMP	CBC-MAC	802.1X



Ondertussen bestaat er ook WPA3, die we op het einde van dit hoofdstuk zullen bespreken. WPA1 en WPA2 zijn echter, historisch gezien, gerelateerd aan WEP en daarom bekijken we deze beiden samen in functie van WEP.

WPA1

Voor de tijdelijke oplossing, WPA1n werden volgende beperkingen geïdentificeerd:

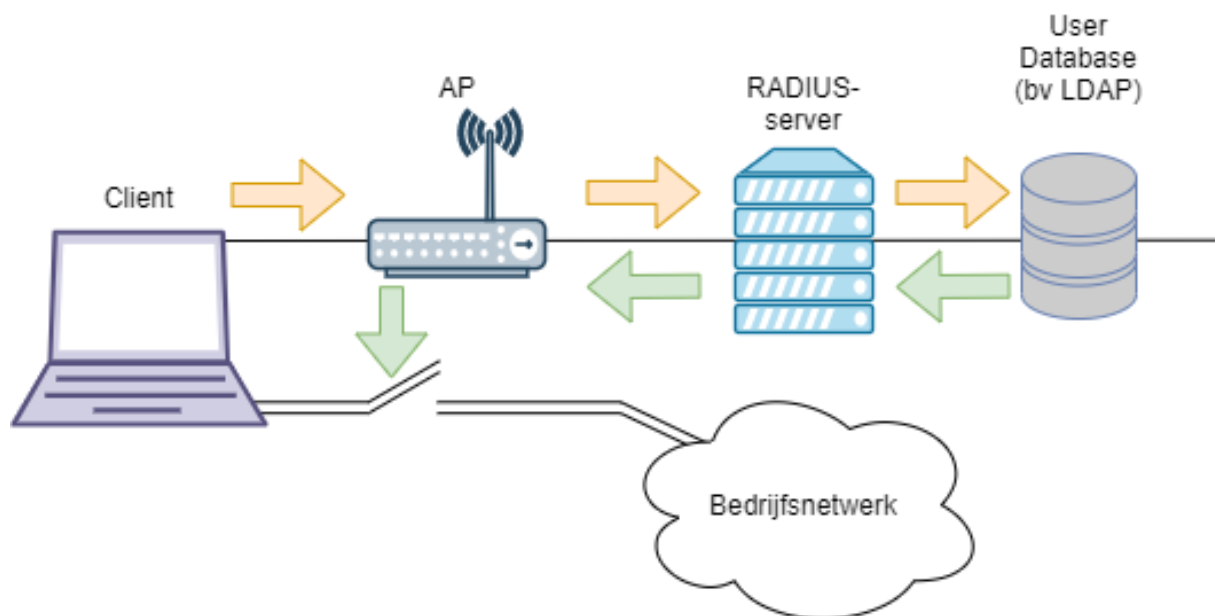
- Zoals verteld, miljoenen WEP-gebaseerde apparaten waren reeds in gebruik. Deze apparaten zouden met behulp van een firmware upgrade gepatcht moeten kunnen worden naar de tussentijdse oplossing.
- De meeste AP's werkten met processoren die reeds quasi volcontinue tegen hun maximum capaciteit werkten. De extra algorithmes die het AP moesten draaien mocht dus maar een beperkte overhead creëren.
- Delen van de RC4 encryptie zijn *hardwired* voor een deel in de hardware van de AP. Hierdoor kunnen bepaalde delen van WEP onmogelijk 'omzeilt' worden en hangen we dus inherent vast aan WEP.

802.1X

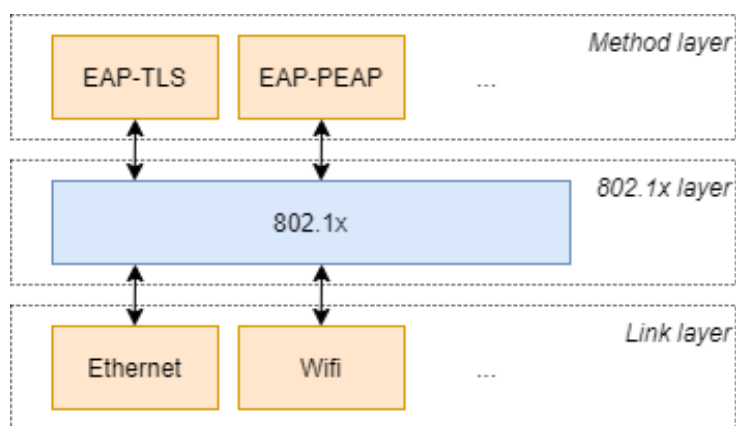
De IEEE 802.1X standaard is, ondertussen, een veelgebruikte standaard in veel draadloze én bedrade netwerken. Het voorziet volgende zaken aan WPA1 en WPA2 netwerken die in *Enterprise-mode* werken:

- (Mutual) **authentication**.
- Een **centraal user management** systeem.
- Een **veilige manier om geheime sleutels uit te wisselen**.

We gaan de volledige werking van de 802.1X standaard hier niet uit de doeken doen, dat zou ons te ver brengen. Het is echter nuttig om te begrijpen dat deze standaard werd gekozen omdat hij ervoor zorgt dat de AP niet meer zelf de authenticatie moeten doen, maar dat ze gebruik maken van de bestaande authenticatie-infrastructuur van het bedrijf. De APs zullen gewoon als een doorgeefluik aan de start tussen gebruiker en de authenticatie-server optreden en de boodschappen tussen beiden uitwisselen. Enkel wanneer het AP toestemming krijgt van de authenticatieserver (meestal een RADIUS server) zal het AP de eindgebruiker toegang tot het draadloze netwerk verschaffen.



802.1X zelf beschrijft niet hoe de authenticatie moet plaatsvinden: het is geen algoritme. Integendeel: het is een **framework** waar binnen andere algorithmen en standaarden, op maat van het bedrijf, kunnen ingeplugd worden. Hierdoor ontstaat een flexibel concept dat bedrijven (of diehard eindgebruikers) niet verplicht om een bepaalde manier van authenticatie (en bijhorende soft-en hardware) te omarmen. 802.1X zorgt voor de vertaling van de authenticatie- boodschappen tussen enerzijds het netwerkprotocol (bv Ethernet, Wifi, maar ook Token Ring, etc.) en de *methode laag*. De methode-laag bevat het te gebruiken authenticatie-protocol en dient **EAP**-compatibel zijn.

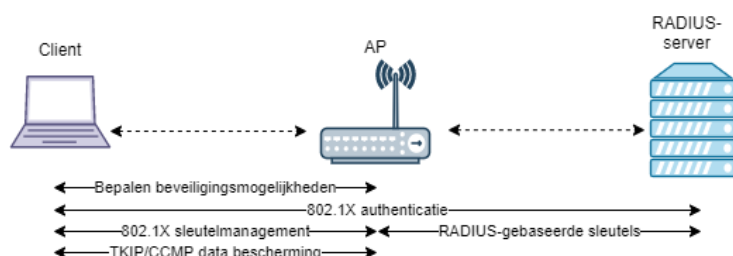


EAP oftewel *Extensible Authentication Protocol* is, zoals de naam doet vermoeden, een uitbreidbaar protocol van te gebruiken authenticatie-methoden. Afhankelijk van de keuze van het bedrijf kan voor een bepaald EAP-protocol gekozen worden, het enige is gebruiksvriendelijker en of veiliger dan het andere. Uiteraard dienen zowel de client als de netwerkinfrastructuur compatibel te zijn met de gekozen EAP-methoden van het netwerk. Koop je dus een AP dat WPA2-Enterprise compatibel is, moet

je nog steeds controleren of het AP compatibel is met de gekozen EAP-methode van het bedrijf.

De meest gebruikte EAP-methoden zijn:

- **EAP-TLS**: gebruikt een TLS-tunnel om op een beveiligde manier te communiceren (we zagen TLS ook reeds aan het einde van crypto waar het gebruikt werd om https-traffic te beveiligen). Hierbij gebeurt een certificaat-gebaseerde authenticatie.
- **EAP-TTLS** (*Tunneled TLS*): Omdat niet alle eindgebruikers zich kunnen authenticeren aan de hand van een certificaat, voorziet PEAP authenticatie met behulp van een username/paswoord login, waarbij wel nog steeds een TLS tunnel wordt gebruikt voor veilige communicatie. Ter info: EAP-TTLS is quasi hetzelfde als *Protected EAP* (PEAP) een ander EAP-protocol dat je soms zal zien passeren.



De voorgaande figuur toont de typische uitwisseling van boodschappen die plaatsvinden wanneer een client voor het eerst verbinding wil maken op een WPA1 of WPA2 Enterprise netwerk:

1. Client en access zoeken samen welke EAP-methoden zij beide kunnen gebruiken om de authenticatie te starten. Enkel indien ze samen tot 1 keuze kunnen komen wordt over gegaan naar stap 2.
2. Vanaf nu zal het AP enkel dienst doen als doorgeefluik tussen de client en de authenticatie-server.
3. Als de server genoeg bewijs heeft om de client te authenticeren zal de server de nodige sleutels genereren en deze aan het AP geven.
4. Het AP zal vanaf nu instaan voor het verdere sleutelbeheer en wanneer nodig de sleutels verversen tijdens de sessie.
5. Wanneer de sleutels tussen client en AP zijn verwerkt krijgt de client toegang tot het netwerk.

In wifi-netwerken met 802.1X worden 2 sets van sleutels aangemaakt:

- Sessie sleutels, ook wel "*pairwise keys*" genoemd: deze is uniek per client en is enkel gekend door die client en het AP.
- Groepsleutels, ook wel "*group keys*" genoemd: deze wordt tussen alle clienten van hetzelfde AP gedeeld en worden gebruikt voor multi-cast traffic.

Indien het *dynamic key exchange* is geconfigureerd dan zal de authenticatie de sessiesleutels eenmalig aanmaken en doorsturen. Daarna zullen specifieke encryptie-sleutels gegenereerd worden bij de

client en AP gebaseerd op deze sessiesleutel. De client (en AP) kan dan zelf, automatisch, op gepaste momenten en nieuwe encryptie-sleutel genereren.

TKIP

Wanneer de nodige sleutels zijn uitgewisseld is het tijd om data te encrypteren. Dit gebeurt door middel van het **Temporal Key Integrity Protocol** (TKIP), wat een suite van algorithms is dat als een wrapper rond WEP wordt gelegd om zo encryptie aan te bieden en daarbij de fouten van WEP minimaliseert.

TKIP omhult WEP met volgende zaken:

1. Een nieuwe **integrity check genaamd Michael** dat een *message integrity code* (MIC) genereert die wél bestand is tegen bitflip aanvallen.
2. Een nieuwe manier van **IV selectie** die replay aanvallen voorkomt.
3. Een **per-pakket sleutel mixing** algoritme dat het probleem met *weak keys* in RC4 oplost.
4. Een *re-keying* mechanisme dat ongeveer elke 10000 pakketjes een nieuwe sleutel doet genereren.

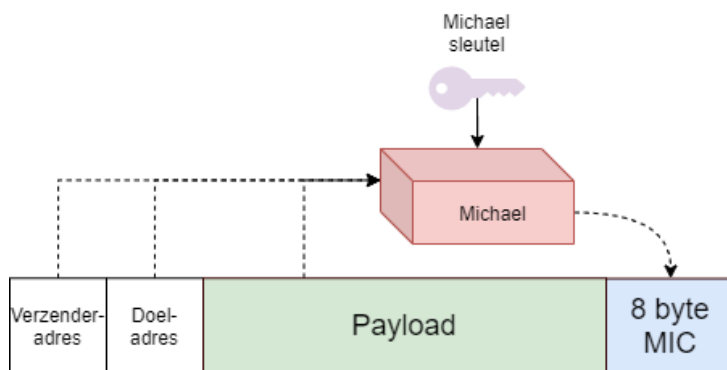
Michael the Mic

Om bewuste aanpassingen aan de payload van een frame te detecteren gebruikt WPA1 een *message integrity check* (**MIC**). Dit is een cryptografisch sterker concept dan de originele CRC checks en wordt verzorgd door het “Michael” algoritme.



In de literatuur wordt meestal gesproken over “Message authentication codes” of MAC’s. Echter, in de IEEE 802 standaarden wordt MAC reeds gebruikt voor *message access control* en werd dus gekozen voor MIC.

“Michael” berekent de MIC van een payload maar gebruikt hierbij ook de authenticatie sleutel, het adres van de verzender én ontvanger. Hierdoor wordt het voor een aanvaller veel moeilijker om een dergelijke MIC na te bootsen, laat staan te *replayen*.



Wanneer TKIP een 2 foute MICs na elkaar detecteert gaat het er van uit dat er een aanval bezig is. Volgende stappen worden dan ogenblikkelijk uitgevoerd door de client:

1. Alle huidige sleutels worden verwijderd.
2. De client verbreekt de verbinding.
3. Er wordt 1 minuut gewacht voor een nieuwe verbinding (*association*) wordt opgezet.

IV selectie verbetering

Om te voorkomen dat fabrikanten weer naïeve oplossingen voor de IV-selectie implementeerden, legde WPA1 nu de regels op. Een ontvangen pakket zal pas aanvaard worden indien het IV van het pakket op het IV van het vorige pakket volgt. Uiteraard zit er een kleine marge om hertransmissies toe te staan, maar een pakket met IV 12933 zal nooit aanvaard worden als het AP vlak ervoor een pakket met IV 4 heeft aangekregen.

Daarnaast wordt ook het IV gevoelig vergroot. TKIP hanteert namelijk een 48-bit IV genaamd de *TKIP sequence counter* (TSC). Deze wordt opgebouwd door de eerste en tweede byte van de originele WEP IV te combineren met 4 bytes van een speciaal gegenereerde *extended IV*.

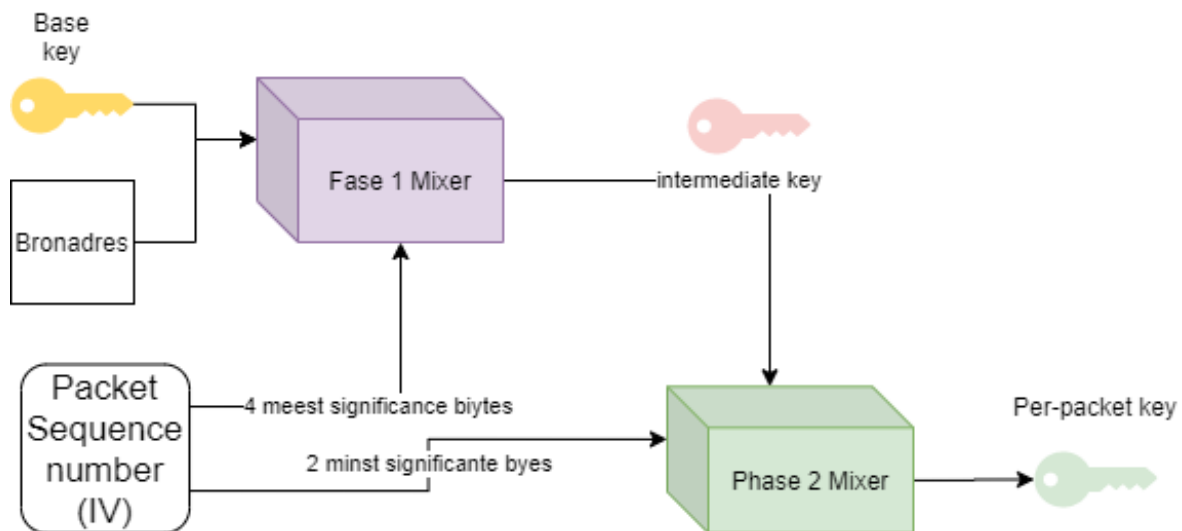
Key mixing en re-keying

Om weak keys te voorkomen gebruikt een TKIP een *key mixing function* dat zal resulteren in een **temporal of per-pakket sleutel** die dienst zal doen als vervanger voor de WEP-sleutel. Deze sleutel zal geregeld ververs worden (vandaar *temporal* oftewel tijdelijk) en heeft dus een beperkte levensduur voor actieve aanvallen.

Het mixen van de sleutel gebeurt in twee fases, waarbij iedere fase een specifieke zwakte van WEP indijkt:

- Fase 1: zorgt ervoor dat alle clients een eigen sleutel hebben doordat het verzender adres (TA, *transmitter address*) wordt toegevoegd aan de basis sleutel.

- Fase 2: zorgt voor een ‘per-pakket’ sleutel waarbij kennis van de IV niet meer door de aanvallers kan misbruikt worden.



Fase 1 mix

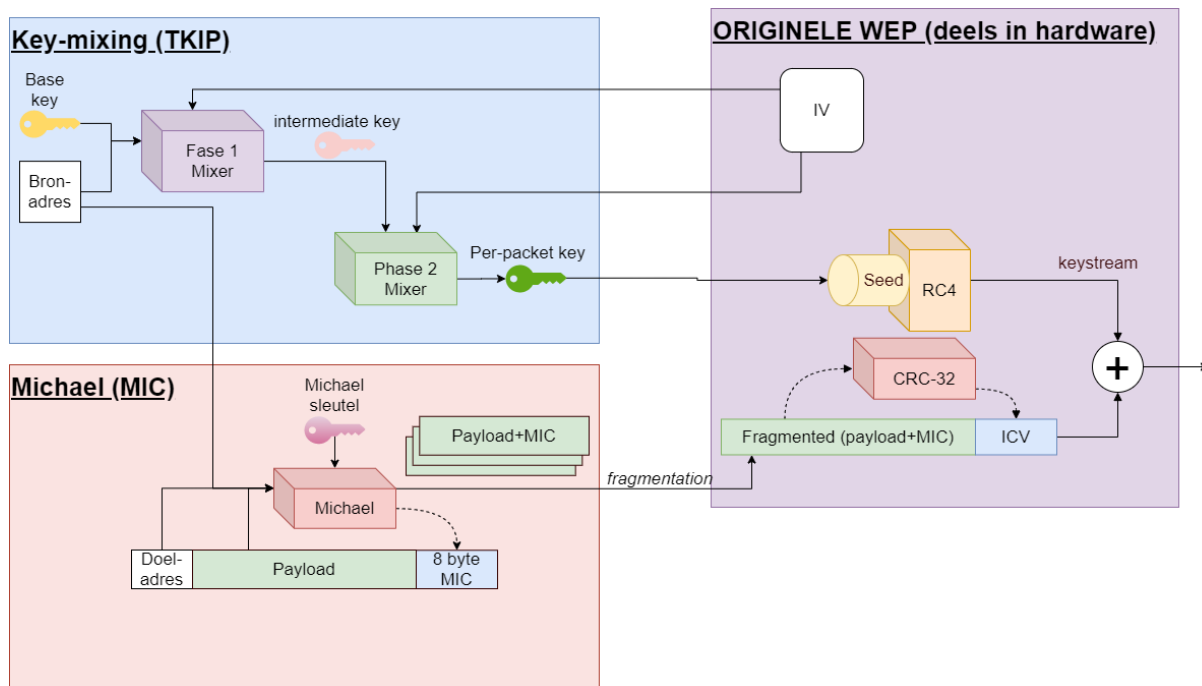
Het MAC address van de client wordt ge-XOR'd met de basissleutel (zijnde ofwel de PSK sleutel in Personal modus of de *temporal* sleutel verkregen van 802.1X tijdens de authenticatie). Dit resultaat wordt door een S-box (substitutie) gestuurd, resulterend in een tussentijdse (*intermediate*) sleutel.

Phase 2 mixing

In deze fase wordt de TSC (de pakket-teller, uitgelegd in de “IV selectie verbetering” sectie) geëncrypteerd samen met de tussentijdse sleutel. De encryptie gebeurt door middel van een kleine Feistel-structuur (zie hoofdstuk crypto) en resulteert in een 128-bit per-pakket sleutel. Vervolgens wordt deze sleutel, samen met delen van de TSC als “WEP-sleutel” en IV aan het originele WEP-gedeelte aangeboden. Hierbij wordt ervoor gezorgd dat er geen RC4 weak keys meer mogelijk zijn omdat die IV worden weggefilterd voor ze aan de hardware worden aangeboden.

WPA1: alle blokjes samen

Finaal kunnen we vervolgens WPA1 visualiseren, waarbij duidelijk is dat we vooral een wrapper rond WEP hebben verkregen, maar dat WEP nog steeds het hart van het systeem is.



In 2009 verschenen er al enkele exploits die WPA1-Personal misbruikten waardoor aanvallers de WPA passphrase (de PSK) konden achterhalen door de handshake aan de start van een sessie te capteren.



Bekijk coWPAtty en Aircrack om WPA1-Personal te *hacken*.

WPA2

De finale oplossing voor het WEP-veiligheidsprobleem werd gegoten in WPA2 waarvan AES het hart vormt. Toen de standaard werd ontwikkeld was AES zonet verschenen en het bleek de ideale toepassing voor WPA2. Hierbij werden 2 onderdelen van AES gebruikt namelijk:

1. AES met *counter mode* voor de encryptie.
2. AES in CBC-MAC mode voor integriteit en authentication.

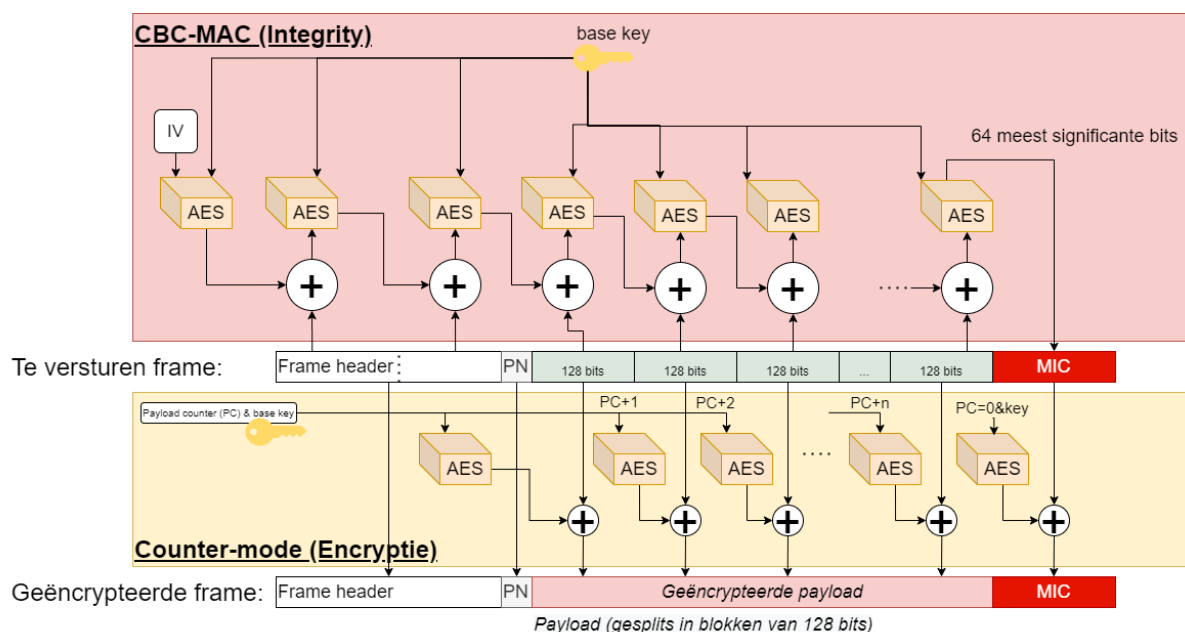
Voor de sleuteldistributie en management werd, zoals vermeld, gebruik gemaakt van de 802.1X standaard.



Toen WPA2 uitkwam ontdekte men dat toch aardig wat bestaande hardware kon gepatch'd worden om ook WPA2-compatibel te zijn. Dit was een opsteker voor velen, daar de originele premise van WPA2 net was dat ze geen rekening zouden houden met de bestaande hardware die reeds op de markt was.

Encryptie en MIC creatie

CCMP gebruikt, net als TKIP, een 48-bit IV die *packet number* (PN) werd genoemd. Deze PN wordt, samen met andere informatie, gebruikt om de AES encryptie van een seed te voorzien. Hierbij wordt het frame (en de header) in blokken van 128 bit verdeeld en zo blok per blok verwerkt. De encryptie gebeurt parallel met het berekenen van de MIC (de ICV in WEP) die finaal achteraan als een laatste blok ook mee wordt geëncrypteerd. Ook nu wordt met een tijdelijke sleutel gewerkt die gebaseerd is op de hoofdsleutel verkregen via 802.11X (enterprise mode) of de passphrase (in personal mode)



In de figuur zie je duidelijk de elegantie van het systeem en hoe de integriteit in parallel wordt berekend met de encryptie. De MIC wordt berekend met behulp van CBC-MAC, wat staat voor **cipherblock chaining - message authentication code**, een veelzeggende naam. Zoals we uit het hoofdstuk crypto weten, zal er bij CBC een keten van encrypties starten, waarbij de encryptie van het huidige blok geXOR'd wordt met het resultaat van de encryptie van het vorige blok. Het finale geëncrypteerde cipherblock zal daarbij dienstdoen als MIC. Bij de minste bitfout ergens in de payload of header zal een totaal andere MIC gegenereerd worden.

Om de payload te encrypteren (merk op dat de header *niét* geëncrypteerd wordt, het wordt enkel mee betrokken om de MIC te berekenen) wordt de **counter mode** van AES gebruikt. Hierbij worden de blokken onafhankelijk van de andere blokken geëncrypteerd en zal een payload counter* (PC) telkens met 1 verhoogd worden en als extra seed voor de AES-motor van het huidige blok dienen (samen met de te gebruiken sleutel).

Helaas, aan alles komt een einde

Tot 2017 ging alles goed. De AES standaard was al jaren een robuuste standaard gebleken en werd op vele plekken nog steeds gebruikt. En dit zou ook bij Wifi zou zijn geweest, waren het niet dat in mei 2017 een Belgische onderzoeker, Mathy Vanhoef, de bevindingen van z'n onderzoek publiceerde. Hij had helaas een belangrijke fout gevonden in de WPA2 standaard. Deze had niets te maken met AES - dat blijft een stevige standaard zijn- maar wel de manier waarop een bepaalde uitwisseling van berichten tijdens de initiële handshake tijdens de authenticatie plaatsvinden tijdens de *sleutel reïnstallatie fase*. Deze aanvallen worden beschreven én gedemonstreerd op krackattacks.com en verplichtten de IEEE om te beginnen werken aan een opvolger voor WPA2.

WPA 3 (“Wifi 6”)

In 2018 kwam de Wifi Alliance uit met de opvolger van WPA2, de titel, je raadt het nooit, was uiteraard WPA3, maar werd ook wel Wifi 6 genoemd. De standaard gaan we hier niet zo gedetailleerd bespreken, het volstaat te begrijpen dat ze

- op veiligheidsniveau state-of-the-art was.
- rekening hield met de noden van 21e eeuwste draadloze netwerken (denk maar aan Internet-of-things apparaten, beveiligde publieke hotspots, etc.).



De Wifi Alliance is in het leven geroepen als een organisatie die ervoor zorgde dat producten wel officieel konden claimen dat hun producten conform een IEEE standaard waren. Enkel wanneer hun producten de nodige tests van de Wifi Alliance aflegden kon het product het label van “Wifi alliance compatibel” product dragen.

Ook in WPA3 werden 2 modes voorzien: een personal en een enterprise mode. Enkele van de interessantste verbeteringen zijn:

- *Simultaneous Authentication of Equals (SAE)*: een nieuw cryptografisch concept waarbij in de personal mode authenticatie veel veiliger kan plaatsvinden dan voorgeen.
- Resistant tegen offline dictionary attacks: iets waar zowel WPA1 en WPA2 last van hadden voor aanvallers met geduld.
- *Forward secrecy*: zelfs als de aanvaller de wifi-sleutel van oude gecapteerde pakketten vindt zal hij deze toch niet kunnen decrypteren. Het aloude “safe now, decrypt later” is dus niet van toepassing op WPA3.
- *Wifi easy connect*: een gebruiksvriendelijke manier om internet-of-things apparaten met het netwerk te verbinden.

- *Wifi enhanced open*: publieke hotspots blijven publiek, maar iedere client heeft z'n eigen veilige kanaal met het AP. Gedaan zijn de dagen van je in de Starbucks zetten om zo prive-traffic van omstaanders te sniffen.
- *Geauthenticeerde encryptie* gebruik maken van “256-bit *Galois/Counter Mode Protocol (GCM-256)*” een cryptocipher dat we hier niet uit de doeken gaan doen.
- Gebruikt de meest secure authenticatie en sleuteldistributie methoden mogelijk binnen 802.1X (HMAC, HMAC-SHA384 en ECDH)

Dankwoord

Een oprechte dank aan Stefaan Somerling (RealDolmen) voor de feedback op het hoofdstuk omtrent GDPR. Geef gerust een sein als u, conform de GDPR wetgeving, liever uw naam niet in dit document ziet staan ;)

Bronnen

- 802.11 Wireless Network (first edition), Gast, O'Reilly, ISBN: 0-596-0018-3
- Meerdere boeken van William Stalling (Computer Security, Cybersecurity, etc.)

